

AutoTraQ's charts and data visualization are built with **Recharts**, which is a chart library made for **React** (and it's based on D3.js under the hood). These charts mostly appear on the **Dashboard page** and are meant to give quick, easy-to-read insights about inventory—things like how stock is trending over time, which parts move the most, which items are sitting too long, and what needs attention right now.

In the project, the main dashboard charts live in `frontend/src/pages/DashboardPage.tsx`. There are also supporting components like `SearchStatsCards.tsx` for showing stats cards tied to search results, and `Skeleton.tsx` for loading placeholders so the page doesn't look empty while data is being fetched. The dashboard includes several visual sections. First are **KPI cards**, which are small summary boxes showing numbers like total parts, total inventory units, total inventory value in dollars, pending requests, and low-stock alerts. Each card includes an icon (from Lucide React) and visual styling to help users quickly understand what's normal versus what might be a problem.

One of the main visuals is the **Inventory History line chart**, which shows how inventory levels change over time. Recharts makes this easier because you build charts by combining React components, like `LineChart`, `XAxis`, `YAxis`, `Tooltip`, and `Line`. The chart is wrapped in `ResponsiveContainer`, which makes it automatically resize to fit different screen sizes (desktop, tablet, mobile). Other dashboard sections include **Top Movers**, which highlights parts with the most stock activity and shows whether the net change is positive (more stock coming in) or negative (more stock going out). There's also **Dead Stock analysis**, which points out items that haven't moved in a long time, helping the team find inventory that might be wasting space or money. The dashboard also shows **Low Stock alerts** for parts that fall below their minimum threshold, plus a **Recent Activity feed** that lists the latest stock-in and stock-out events with details like who did it, what changed, and when it happened.

The data for these charts follows a simple flow: the frontend calls the **backend API**, the results come through the app's API client, the dashboard stores the results in React state, and then Recharts uses that state data to render the visuals. While data is loading, the app shows **skeleton loading components** (placeholders shaped like the real UI) instead of blank space or a spinner, which makes the experience feel smoother and more polished.

For meeting talking points, the main ideas are: Recharts was chosen because it's **React-friendly** and easier to use than writing raw D3 code; the dashboard acts like an "inventory control center" where users can quickly see health, trends, and problems; the history chart helps spot patterns over time; Top Movers helps with forecasting and reorder decisions; Dead Stock helps identify wasted inventory; skeleton loading improves the UI experience; everything is responsive using

ResponsiveContainer; and the chart data comes from the same REST API as the rest of the app, keeping a single source of truth without needing a separate analytics system.